

COMP 3004 Fall 2025

HinLIBS Deliverable 1

Nov 16 - 2025

Team # 31:

Megan Pare - 101292260

Shawn Maxwell - 101306587

Zainab Baburi - 101311391

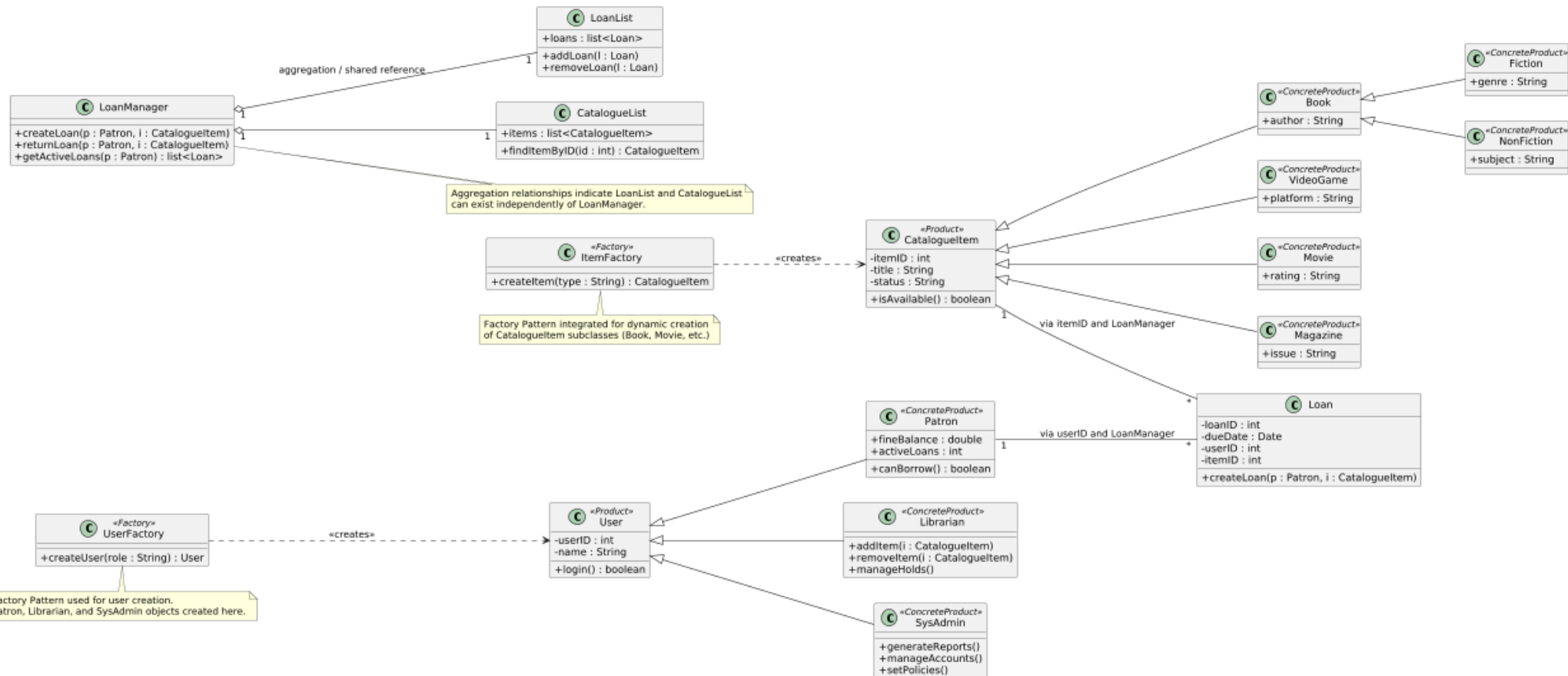
Ryan Anokye - 101159370

Part 1 – Design Pattern and Class Diagram

1. Design Pattern: Factory

2. UML Class Diagram

Figure 1 - UML Class Diagram showing Factory Design Pattern and LoanManager Integration in HinLIBS



3. Written Justification:

The Factory Design Pattern exists to maintain a clean and extensible code base of how to make what domain objects (i.e. catalogue items, user accounts) that build the software essentially. For example, ItemFactory allows creation of subclasses of CatalogueItem (Book, VideoGame, etc.) dynamically and UserFactory handles roles and subclasses of user accounts (Patron, Librarian, SystemAdmin).

By introducing the Factory Pattern, the question of when/how these complex builds should occur was answered while reducing the complexity need for borrow/catalog/login since not every section that wanted to borrow/catalog/login to itself was able to use a if or switch statement logic to determine which subclass needed to build/return or set. For each controller was the same, as were various error messages along the way. Instead, all logic could be contained within a factory and the UI windows (i.e. window_home.cpp, window_librarian.cpp, window_sysadmin.cpp) or control classes could ask what it needed without concern of how it was built from such patterns.

Furthermore, the Factory Pattern assists in maintainability and extensibility goals of the project:

Maintainability: Object types and object attributes require small changes (found only in the factory class created). Object types/attributes do not require changes across the system.

Extensibility: New media types/new types of users can be made via a similar approach as those already existing Factory Patterns without extending any system client calls (for all other modules have standardized needs).

Consistency: Different ways of constructing the same object/partially constructed objects due to complicated logic working across modules are avoided.

Encapsulation: Good OOP with per COMP 3004 class has a modular design that a factory limits what the builders need to know about their constructors.

Ultimately, the Factory Pattern allows for increased flexibility and growth with a small percent of code spread throughout any of the associated subsystems with the system (borrowing, cataloging, administration).